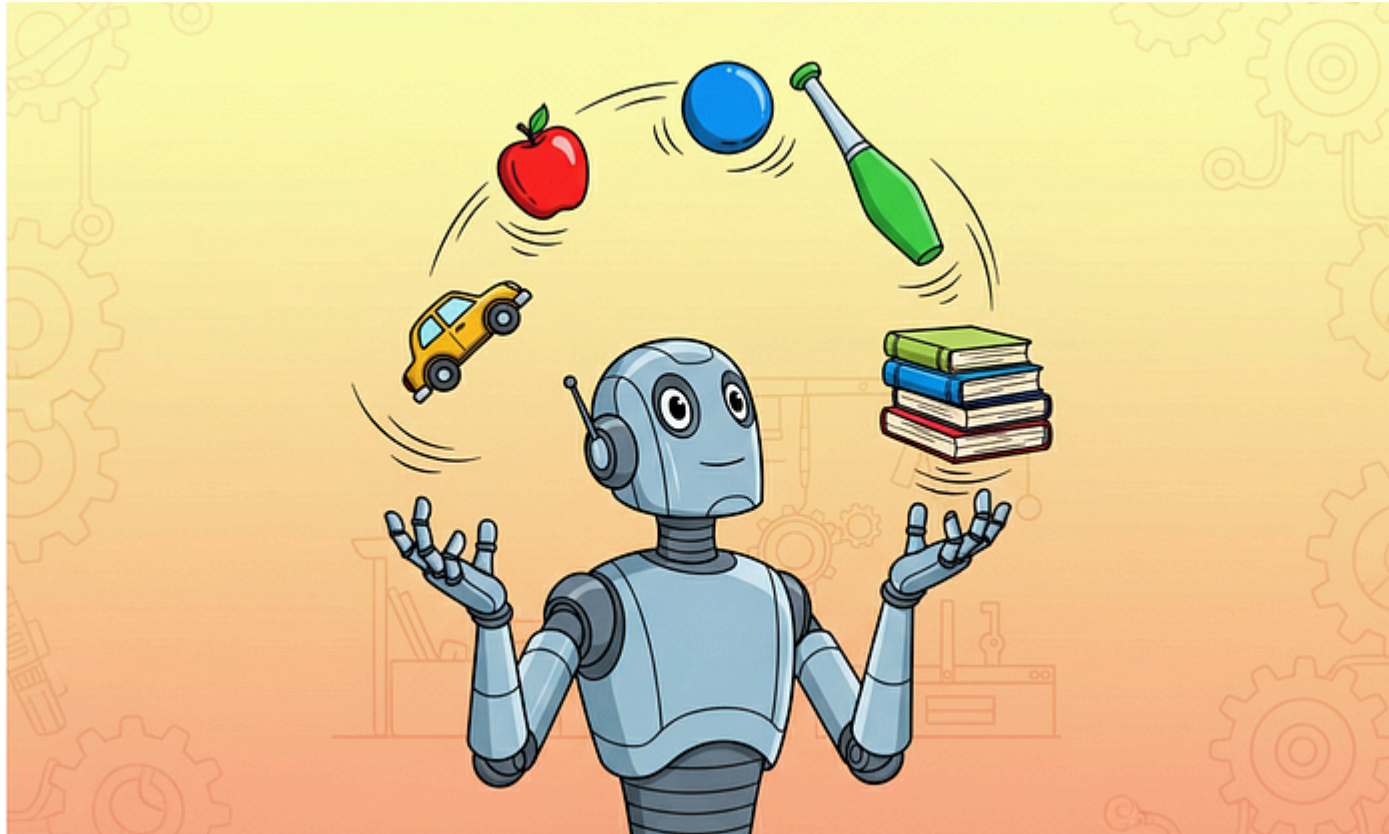


[< Go to the original](#)



Control is All You Need: Why Most AI Systems & Agents Fail in the Real World, and How to Fix It

**Kenny Vaneetvelde**

Follow



Towards AI a11y-light ~9 min read · September 8, 2025 (Updated: September 8, 2025) · Free: No

If you've spent any time on social media the past few months, you've been sold a very specific dream about AI agents. You've seen the demos: an "*autonomous*" agent that builds a complete Snake game from a single prompt (*Wowzers!*), or an entire "*research crew*" that generates an amazing comprehensive report on a complex topic that in reality **some times** performs wonderfully, some times outputs **trash**, and **more often than not** doesn't give you 100% exactly what you were expecting. There is this compelling vision of digital employees handling complex tasks with little to no human intervention.

NOTE: If you do not have a Medium account, [use this URL](#) to read the full article, no catch.

But for those of us who have spent years in the software development trenches, building **real-world systems**, this dream feels **dangerously disconnected** from

libraries that try to present AI development as much more of a new paradigm than it really is, I've had quite enough. **I want to see this industry thrive, not go in the wrong direction.**

Those cool demos are an illusion.

They hide a fundamental lack of control and reliability that is **simply unacceptable** for any serious application. The AI industry is in a frantic rush to invent a brand-new, overly complex discipline called "*AI Agent Engineering*," complete with its own proprietary paradigms and opaque tools. **This is a profound mistake.** The future of AI isn't about chasing a **fantasy of full autonomy**; it's about **building systems that work predictably**, and that means **putting the developer firmly back in control.**

The Autonomy Trap

The core issue with the current hype cycle is that it **champions a "black box" approach** to AI. Frameworks that promise **full autonomy** encourage you to chain together multiple agents, give them a goal, and just trust that the magic will

But in a production environment, **"magic" is another word for "un-debuggable."** Here's what I've seen time and again when this approach hits the real world:

- **Unpredictable Costs and Performance:** Autonomous chains often make hidden, recursive calls to LLMs. A simple query can easily spiral into dozens of expensive API calls, burning through your budget with no clear benefit.
- **Deceptive Demos:** The "AI that builds a Snake game" demo is impressive until you realize **the model has been trained on thousands of examples of that exact problem**. Ask one of these "autonomous" systems to build something **genuinely new — something your business actually needs** and that isn't plastered all over its training data — and **the illusion shatters *completely***. These successes are **often cherry-picked** after dozens of failed attempts.
- **Chaos Instead of Collaboration:** When you let multiple agents converse **without explicit control**, you invite chaos. They get stuck in loops, misunderstand each other, and produce **wildly inconsistent** results. This is **not a stable foundation** for a business process.

This isn't just a technical problem; **it's a philosophical one**. The challenges of AI, like non-determinism, **don't require us to invent entirely new programming paradigms**. On the contrary, they demand a *more rigorous application* of our

need to learn a completely new, "database-native" way of thinking just to use a new database, abandoning all your knowledge of *SQL*, *ORMs*, and *transaction management*.

The Sobering Reality: An AI Agent is Just a Powerful Function Call

If we strip away the hype, what is an AI agent at its core? It's a non-deterministic function. It takes structured data (your prompt and schemas) as input and returns structured data as output. That's it.

Once you see it this way, everything changes. You realize that **you don't need a new discipline to manage it**. You just need the discipline you already have. Because it's **just a function call**, all the principles we've honed over decades don't just apply — they **become more critical than ever**:

- **Modularity and Single Responsibility:** Why build one monolithic, "do-everything" agent when you can compose a solution from small, testable functions that each do one thing well?
- **Clear Interfaces (Contracts):** This is the role of schemas. A well-defined input and output schema is the API contract for your AI function.

we've always used. The logic that decides which agent or tool to call next belongs in your code, where you can control it, test it, and log it.

This is the path to building robust systems. You lean on the deterministic, reliable nature of traditional code to manage and contain the non-deterministic, powerful capabilities of the AI.

Schemas: The Unsung Heroes of Reliable AI

This brings us to the most crucial, yet often overlooked, aspect of building with control: **schemas**. Most people just "prompt and pray," engineers spend countless hours tweaking natural language instructions, hoping the LLM will finally give them the output they need.

This is not engineering; it's guesswork.

A schema-first approach completely flips this on its head. By defining a strict Pydantic schema for your agent's input and output, you are doing much more than just validating data. That schema, complete with its field names, types, and descriptions, gets serialized and becomes a core part of the prompt itself. It does a huge amount of the heavy lifting.

microservices. We agree on a schema for passing data, and as long as both sides adhere to that contract, the system works. We can apply the same sanity to AI. Instead of putting all our effort into the perfect natural language prompt, we define a perfect data contract. The schema guides the LLM toward the desired structure, dramatically increasing reliability and making the entire interaction far more predictable.

Building with Control: The Agentic Pipeline in Practice

This philosophy naturally leads to a more pragmatic and powerful architectural pattern: the agentic pipeline.

An agentic pipeline is a system where a small, single-purpose AI agent acts as a crucial cog in a machine that is otherwise controlled by **explicit, predictable code**. The application might be 90% traditional software and only 10% AI, but that 10% unlocks capabilities that were previously impossible.

Let's paint a clearer picture than the flashy demos. Imagine a workflow for processing an incoming invoice:

2. It uses a **battle-tested** library like `PyPDF2` to extract the raw text. No AI needed yet.
3. It then makes a single, controlled call to an AI agent. It passes that text along with a strict schema that asks for three specific things: the vendor name, the total amount, and the due date.
4. The agent returns a **structured JSON object** that is guaranteed to match your schema.
5. Your Python script then validates the *values* within this object. Is the date in the correct format? Is the amount a valid number?
6. Finally, the script uses a standard database library, like `SQLAlchemy`, to save the validated data to your database.

In this workflow, the AI is a specialized, schema-driven step in the assembly line, not the entire factory. You, the developer, remain the orchestrator. You control the flow, handle the errors, and ensure the integrity of the process. This is how you build production-ready AI.

Atomic Agents: A Return to Engineering Sanity

wanted a tool that was intentionally boring — **a framework that doesn't invent new paradigms** but instead provides a **clean, minimal structure for applying the ones that already work.**

Atomic Agents is built from the ground up on a few simple, powerful ideas that should feel familiar to any experienced engineer:

- **Atomicity:** Every agent has one job and does it well. This makes them easy to test, debug, and reuse.
- **The IPO Model:** Every component has a clear Input, a clear Process, and a clear Output, enforced by schemas. No more guessing what data is flowing through your system.
- **Developer-Centric Design:** It's just Python. You can use your existing debugger, your existing logging tools, and your existing engineering practices. No special "AI-native" complexity needed.

Let's look at a quick example. This is how you'd build a simple agent.

Python

Copy

```
from atomic_agents.agents.atomic_agent import AtomicAgent
from atomic_agents.context.chat_history import ChatHistory
from atomic_agents.context.system_prompt_generator import SystemPromptGenerator
from atomic_agents.base.base_io_schema import BaseIOSchema
from pydantic import Field

# Define the "API contract" for your agent
class SimpleInput(BaseIOSchema):
    user_query: str = Field(..., description="The user's question")

class SimpleOutput(BaseIOSchema):
    agent_response: str = Field(..., description="The agent's answer")
    followup_questions: list[str] = Field(..., description="A list of suggested followup questions")

# Initialize the agent with explicit configuration
basic_agent = AtomicAgent(
    client=your_configured_client, # Your configured LLM client
    model="gpt-5-mini",
    input_schema=SimpleInput,
    output_schema=SimpleOutput,
    chat_history=ChatHistory(),
    system_prompt_generator=SystemPromptGenerator(
        background={"persona": "You are a helpful assistant."}
    )
)

# Run the agent with structured input
```

Notice how this isn't some new language or abstract concept. **It's an object with a clear configuration.** We define our 'API contract', which is sent to the LLM alongside the system prompt, with `BaseIOSchema`, and we call it with a method. It's just good, clean object-oriented design applied to an LLM call. This also allows you to take more "weight" and dependence off of the system prompt, and spent it just on good architecture and schema definitions.

Conclusion: Build Systems, Not Just Demos

The AI revolution **doesn't demand** that we abandon decades of software engineering wisdom. On the contrary, **it demands we apply it more rigorously than ever.** The most powerful and reliable AI systems **won't be built by those who chase the magic of new paradigms, but by disciplined engineers who know how to wield a powerful new tool within the sturdy framework of practices they've spent years perfecting.**

The future of production-grade AI isn't about giving up control for the illusion of autonomy. It's about harnessing the power of AI with the precision and discipline

So, if you're tired of wrestling with frameworks that fight you every step of the way, I urge you to start thinking differently. Start prioritizing control.

- **Explore Atomic Agents on GitHub:** <https://github.com/BrainBlend-AI/atomic-agents>
- **Join the discussions on the Atomic Agents subreddit or the Discord Server** and start building AI agents the *right* way.

It's time to build smarter, not harder. It's time to build atomically.

Ready to transform your AI vision into a market-ready reality?

We're BrainBlend AI, a team of veteran software engineers with over 15 years of experience building robust, scalable applications. We partner with you to create high-quality AI solutions that aren't just fast to market, but built to last.

Whether you need end-to-end project development, strategic workshops, or to augment your existing team, we provide the expertise to make it happen.

Linkedin

If you **loved** my content and want to **get in touch**, you can do so through **LinkedIn** or even feel free to reach out to me by email at **kenny@brainblendai.com**

If you need an AI-driven project or prototype developed, please contact my agency: **BrainBlend AI** and we will make sure your project gets the quality treatment it deserves in a way that is maintainable and ready for production!

You can also find me on **X/Twitter** or you can give me a **follow on GitHub** and check out and star any of my projects on there, such as **Atomic Agents!**

Freedium

ko-fi librepay patreon



#artificial-intelligence #python #ai #programming #coding